

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

# College of Computing and Data Science

# SC2006 – Software Engineering

# Lab 4 Deliverables

### SCSJ Group 3



| Name | Matriculation No. |
|---|---|
| Xu Hanwen | U2522613H |
| Teo Kok Wei, Oscar | U2410894D |
| K Eswaran | U2521677A |
| Mao Yikai | U2520028F |
| Tan Leong Jun Joseph | U2321339H |

## Work Products and Authors

The author(s) of each Lab 4 work product are listed below.

| Work Product | Author(s) |
|---|---|
| Working application prototype and source code | SCSJ Group 3 (each change's author is recorded in the repository's commit history) |
| Black-box and white-box test cases and results | Tan Leong Jun Joseph |
| Demo script | Tan Leong Jun Joseph |

## Working Application Prototype

Crowdy is a working prototype built with Expo / React Native and a Node.js / Express backend. It runs in Expo Go (SDK 57) on a phone connected to the same Wi-Fi network as the backend. iOS is our demo platform, since Expo Go no longer supports Google Maps on Android; the rest of the app also works on Android, without the map.

The prototype implements every use case from our Lab 3 design, plus friends and a meetup planner, which were added after Lab 3:

| Feature | What it does | Use Case(s) |
|---|---|---|
| Live crowd map | MRT/LRT crowd levels from LTA DataMall and mall crowd levels from Google Popular Times, as colour-coded pins with a rail-line layer | AggregateCrowdData (#1-1), ViewNearbyVenues (#2-1), ClusterMapMarkers (#2-2), ToggleMapLayer (#2-3), RecentreMap (#2-4) |
| Right now card | How many malls are quiet, moderate, busy or closed, the crowd level near you, and the quietest open mall nearby | ViewCrowdSnapshot (#2-5) |
| Venue detail | Current crowd level, the typical week as an hourly chart, and opening hours | ViewVenueDetail (#3-1) |
| Nearby alternatives | Up to three less crowded venues of the same type within 3 km | ViewAlternatives (#3-2) |
| In-app directions | Transit, walking, driving and cycling routes from OneMap, with the live crowd level at each station you board | GetDirections (#3-3) |
| Crowd heatmap | An overlay where busy malls and stations glow, so hotspots stand out (iOS) | ViewCrowdHeatmap (#4-1), GenerateHeatmapImage (#4-2) |
| Search | Find any venue by name, address or postal code, forgiving small typos | SearchVenue (#5-1) |
| Accounts and favourites | Sign up and log in; save venues as a guest or synced to an account | AccountAuthentication (#6-1) to LogOut (#6-6) |
| English / 中文 | The whole app in English or Simplified Chinese | ChangeLanguage (#7-1) |
| Friends | Add friends by username, answer requests, and set a home station that only friends can see | New since Lab 3 (to be added in the Lab 5 SRS) |
| Meetup planner | For 2 to 6 people, the three best malls to meet at, now or at a chosen time in the next week, by fairness, total travel time or crowd level | New since Lab 3 (to be added in the Lab 5 SRS) |

## Source Code

The full source code is in our team repository at **http://10.96.189.23/sc2006/scsj-h3**. The main branch holds the version we submit; development happens on the dev branch, and members' feature branches are merged into dev after review.

The code follows the folder structure described in Lab 3. The mobile app (app/) and the backend server (server/) are separate. Boundary classes are in screens and components, control classes are in services, routes and context, and entity classes are in types. Key classes and public methods are documented with JSDoc comments.

Since Lab 3, the code has grown to include the friends and meetup features (FriendsScreen and MeetupScreen in the app; the /friends and /meetup routes and the meetup service on the server), and per-client rate limiting on sign-in, meetup searches and friend requests.

Both halves have automated tests, run with npm test: 47 tests in server/ and 24 in app/, all passing. The test cases in the next section are part of these suites.

## Test Cases and Testing Results

We tested one control class with black-box techniques (equivalence classes and boundary values), and two methods with white-box basis path testing. Every test case is automated with Node.js's built-in test runner (node:test). The actual outputs below are what the tests printed when they ran, and each section names its test file.

### Black Box Testing: AlternativesFinder

Class under test: **AlternativesFinder** (server/src/services/alternatives.ts), the control class behind ViewAlternatives (#3-2). Its method findAlternatives(venue, allVenues) returns up to three venues that are the same category, open with a crowd reading, less crowded than the venue being viewed, and within 3 km, closest first.

Test file: server/tests/alternatives.blackbox.test.ts. In every test case the viewed venue is a mall at 60% (Moderate). Each candidate is the same category, Low at 30% and 1 km away, unless the test input says otherwise.

#### Equivalence classes

| Input Condition | Valid Classes | Invalid Classes |
|---|---|---|
| Candidate's category | E1: same as the viewed venue | E2: different (e.g. a station for a mall) |
| Candidate's crowd status | E3: open with a reading (Low to Very High) | E4: Closed<br>E5: no reading (Unavailable) |
| Candidate's crowd % | E6: lower than the viewed venue's | E7: equal to it or higher |
| Distance from the viewed venue | E8: up to 3 km | E9: more than 3 km |
| Number of qualifying candidates | E10: 1 to 3 (all returned)<br>E11: more than 3 (closest 3 returned) | E12: none (nothing shown) |

#### Boundary values

| Boundary | Values Tested | Expected |
|---|---|---|
| Viewed venue's crowd of 60% | Candidate at 59%, 60%, 61% | Only 59% is returned |
| Distance limit of 3 km | Candidate at 2.999, 3.000, 3.001 km | 2.999 km and 3.000 km are returned |
| Result limit of 3 | 3 and 4 qualifying candidates | 3 returned in both cases, closest first |

To keep the number of test cases small, one case covers all valid classes together (BB-01), one case covers each invalid class (BB-02 to BB-05), and the boundary cases (BB-06 to BB-13) also cover classes E6 to E9, E10 and E11.

**Test cases and results**

Outputs are the ids of the returned venues, closest first; [ ] means no alternatives are shown.

| ID | Test Input | Expected Output | Actual Output |
|---|---|---|---|
| BB-01 | Same category, Low 30%, 1 km (E1, E3, E6, E8, E10) | [c] | [c] |
| BB-02 | Candidate is a station (E2) | [ ] | [ ] |
| BB-03 | Candidate is closed (E4) | [ ] | [ ] |
| BB-04 | Candidate has no reading (E5) | [ ] | [ ] |
| BB-05 | No other venues (E12) | [ ] | [ ] |
| BB-06 | Candidate at 59% | [c] | [c] |
| BB-07 | Candidate at 60% | [ ] | [ ] |
| BB-08 | Candidate at 61% | [ ] | [ ] |
| BB-09 | Candidate 2.999 km away | [c] | [c] |
| BB-10 | Candidate 3.000 km away | [c] | First run: [ ] (defect)<br>After fix: [c] |
| BB-11 | Candidate 3.001 km away | [ ] | [ ] |
| BB-12 | 3 valid candidates at 2, 0.5 and 1 km | [c0.5, c1, c2] | [c0.5, c1, c2] |
| BB-13 | 4 valid candidates at 2, 0.5, 1 and 1.5 km | [c0.5, c1, c1.5] | [c0.5, c1, c1.5] |

**Defect found:** BB-10 failed on its first run. Floating-point rounding put a venue exactly 3 km away at 3.000000000000002 km, so it was left out, although the requirement says "within 3 km". We changed findAlternatives to round each distance to the nearest metre before comparing it, and all 13 cases now pass.

## White Box Testing: isOpenAt

Method under test: **isOpenAt(hours, at)** in server/src/services/openingHours.ts. It decides whether a venue is open at a given moment from its listed opening hours, including hours that run past midnight and venues open round the clock. It returns true, false, or null when the hours can't be read. PopularTimesService uses it so that a mall shows as Closed outside its opening hours (AggregateCrowdData (#1-1)), and the meetup planner uses it for a chosen meeting time.

## Control flow graph

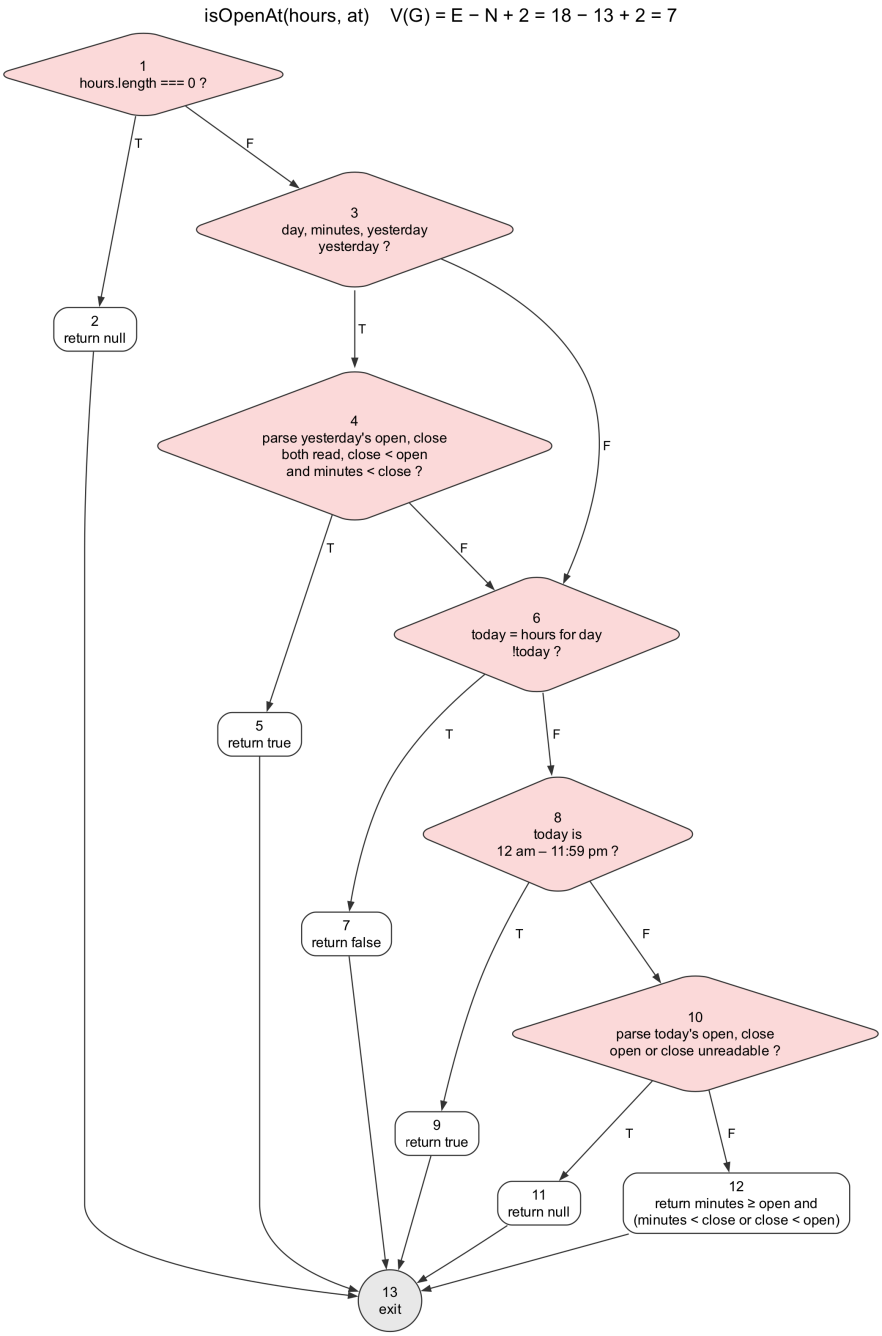



*Figure 1. Control Flow Graph of isOpenAt*

Each node is numbered. Diamonds are decisions, with true (T) and false (F) branches. Each compound condition, such as whether yesterday's hours run past midnight, is one decision node.

### Cyclomatic complexity

V(G) = E − N + 2 = 18 − 13 + 2 = 7. As a check, the number of decision nodes + 1 = 6 + 1 = 7 (nodes 1, 3, 4, 6, 8 and 10). The basis set therefore has 7 independent paths.

### Basis paths

| Path | Nodes | What It Exercises |
|---|---|---|
| P1 | 1-2-13 | No opening hours listed |
| P2 | 1-3-6-7-13 | No hours for yesterday or today |
| P3 | 1-3-4-5-13 | Still open on yesterday's hours, which run past midnight |
| P4 | 1-3-4-6-7-13 | Yesterday's hours don't reach today, and no hours today |
| P5 | 1-3-4-6-8-9-13 | Open round the clock today |
| P6 | 1-3-4-6-8-10-11-13 | Today's hours can't be read |
| P7 | 1-3-4-6-8-10-12-13 | Normal hours today |

### Test cases and results

Test file: server/tests/openingHours.basisPath.test.ts. Every test is on Sunday 18 October 2026, so "yesterday" is Saturday.

| Path | Test Input | Expected Output | Actual Output |
|---|---|---|---|
| P1 | hours: none; at: Sunday 3 pm | null | null |
| P2 | hours: Wednesday 10 am – 10 pm only; at: Sunday 3 pm | false | false |
| P3 | hours: every day 6 pm – 2 am; at: Sunday 1 am | true | true |
| P4 | hours: Saturday 10 am – 10 pm only; at: Sunday 3 pm | false | false |
| P5 | hours: every day 12 am – 11:59 pm; at: Sunday 3 am | true | true |
| P6 | hours: every day "Open 24 hours" (unreadable); at: Sunday 3 pm | null | null |
| P7 | hours: every day 10 am – 10 pm; at: Sunday 3 pm | true | true |

All 7 basis paths pass, and together they run every edge of the graph.

## White Box Testing: tokenHits

Method under test: **tokenHits(queryToken, haystackTokens)** in app/src/services/venueSearch.ts, part of SearchController in our design (SearchVenue (#5-1)). For one word the user typed, it looks through a venue's name or address words for the same word, a word

starting with it, or a word one typo away (for words of four or more letters). It returns "exact", "prefix", "fuzzy" or null. Exact and prefix matches return straight away; a fuzzy match is only returned if no word starts with the query.

### Control flow graph

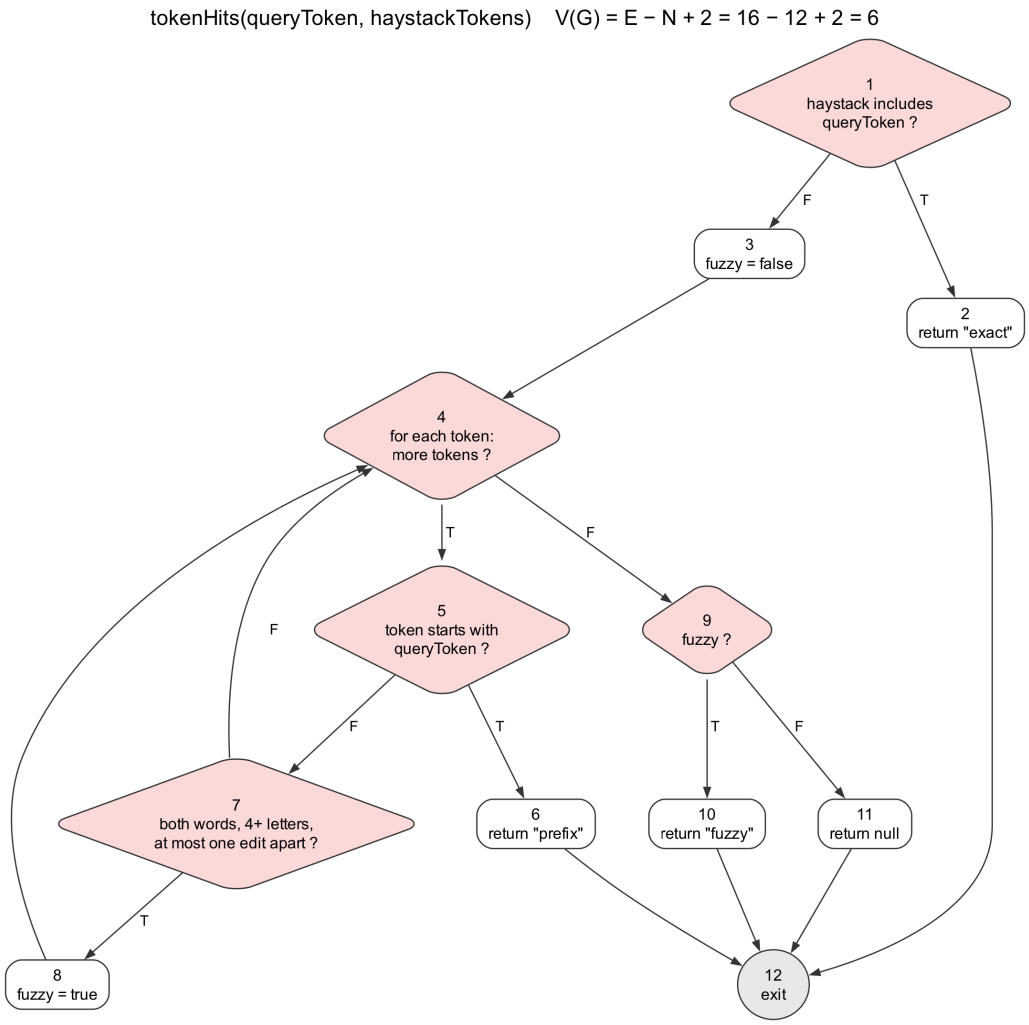



*Figure 2. Control Flow Graph of tokenHits*

Node 4 is the loop over the venue's words; the false (F) branch of node 7 goes back to it for the next word.

### Cyclomatic complexity

V(G) = E − N + 2 = 16 − 12 + 2 = 6. As a check, the number of decision nodes + 1 = 5 + 1 = 6 (nodes 1, 4, 5, 7 and 9). The basis set therefore has 6 independent paths.

### Basis paths

| Path | Nodes | What It Exercises |
|---|---|---|
| P1 | 1-2-12 | The query is one of the words |
| P2 | 1-3-4-9-11-12 | No words to match (the loop runs zero times) |
| P3 | 1-3-4-5-6-12 | A word starts with the query |
| P4 | 1-3-4-5-7-4-9-11-12 | A word that doesn't match at all |
| P5 | 1-3-4-5-7-8-4-9-10-12 | A word one typo away |
| P6 | 1-3-4-5-7-8-4-5-6-12 | A word one typo away, then a word starting with the query (prefix wins) |

### Test cases and results

Test file: app/tests/venueSearch.basisPath.test.ts.

| Path | Test Input | Expected Output | Actual Output |
|---|---|---|---|
| P1 | query: "orchard"; words: orchard, road | "exact" | "exact" |
| P2 | query: "orchard"; words: none | null | null |
| P3 | query: "orch"; words: orchard | "prefix" | "prefix" |
| P4 | query: "bugis"; words: orchard | null | null |
| P5 | query: "orchrd"; words: orchard | "fuzzy" | "fuzzy" |
| P6 | query: "vivo"; words: viva, vivocity | "prefix" | "prefix" |

All 6 basis paths pass, and together they run every edge of the graph.

# Demo Script

Our demo follows the Lab 5 manual and takes about 20 minutes: about 15 minutes of live demo, 2 to 3 minutes on good software engineering practices and our system design, and 2 to 3 minutes showing traceability for two use cases. One member drives the main phone, mirrored to the screen; a second member uses another phone for the friends and meetup scenario. Before the demo, the backend is running with the shared crowd data synced, and two test accounts exist that are not yet friends.

Separately, we will record a 5 to 7 minute demo video of the app for the Lab 5 submission.

## Demo Plan and Timing

| Time | Presenter | What We Show |
|---|---|---|
| 0:00 – 1:00 | K Eswaran | The problem, our users (commuters, residents and the general public), and the use case diagram |
| 1:00 – 6:00 | Xu Hanwen | Live demo, scenario 1: a commuter deciding where to go |
| 6:00 – 8:30 | Mao Yikai | Live demo, scenario 2: a regular user with an account |
| 8:30 – 14:00 | Teo Kok Wei, Oscar, Tan Leong Jun Joseph | Live demo, scenario 3: friends planning a meetup, on two phones |
| 14:00 – 16:30 | Tan Leong Jun Joseph | Good software engineering practices and system design, with the architecture diagram |
| 16:30 – 19:00 | [Name] | Traceability for ViewAlternatives (#3-2) and SearchVenue (#5-1) |
| 19:00 – 20:00 | All | Questions |

## Scenarios by End User

### Scenario 1: A commuter deciding where to go (guest)

1. Open Crowdy. The Right now card shows how many malls are quiet, moderate, busy or closed, and the quietest mall nearby (ViewCrowdSnapshot (#2-5)).
2. Turn on the transit layer: stations are coloured by LTA's live crowd levels. Turn on the heatmap to show the island's hotspots (ToggleMapLayer (#2-3), ViewCrowdHeatmap (#4-1)).
3. Open a busy mall: its crowd level now, its typical week (tap an hour on the chart) and its opening hours (ViewVenueDetail (#3-1)).
4. Scroll to the quieter places nearby and open one within 3 km (ViewAlternatives (#3-2)).
5. Tap Directions: the transit route is drawn on Crowdy's map with the crowd level at the boarding station; switch to walking (GetDirections (#3-3)).

6. On the Search tab, find a mall by name, by postal code, and with a typo such as "orchrd" (SearchVenue (#5-1)).

**Scenario 2: A regular user with an account**

1. As a guest, save a mall to favourites.
2. In Settings, sign up. Crowdy asks whether to bring the guest save into the new account (CreateAccount (#6-3), ManageFavourites (#6-4)).
3. Log in on the second phone: the saved mall is there too (LoginAccount (#6-2), ViewSavedVenues (#6-5)).
4. Switch the language to 中文 and turn on dark mode; both are remembered after the app restarts (ChangeLanguage (#7-1)).

**Scenario 3: Friends planning a meetup (two phones)**

1. Phone 2 sends a friend request to phone 1's username; phone 1 accepts it.
2. Each sets a home station, for example Jurong East and Tampines.
3. On phone 1, open Meetup: add yourself, add the friend, and add a third person by picking Woodlands.
4. Choose "Fairest for everyone" and "Now": the top three malls appear with each person's travel time and the crowd level.
5. Change the time to Saturday 7 pm and the priority to "Least crowded": the suggestions change to match that time's usual crowds and train and bus timetable.

## Highlights: Innovative Solutions and Good Software Engineering

**Innovative solutions**

- Mall crowd levels from Google Popular Times, which has no official API: one machine collects them weekly and shares them with every backend through the database.
- Crowd-aware directions: the live crowd level at each MRT/LRT station you board.
- A meetup planner that ranks malls for a whole group by fairness, total travel time or crowd level, now or at a chosen time in the next week, using each mall's usual crowd at that hour and that time's timetable.
- A crowd heatmap where only busy places glow, so hotspots stand out.
- The whole app in English and Chinese.

**Good software engineering practices**

- A client–server architecture with boundary, control and entity classes, traceable from the Lab 3 design class diagram to folders and files.
- Design patterns: Facade (CrowdAggregator, ApiClient), Adapter (the LTA, OneMap and Google Maps clients), Singleton (BrowserManager, DatabasePool), Repository (PopularTimesStore) and Observer (the session and language contexts).
- 71 automated tests, including the black-box and white-box cases above. A boundary value test found a real defect, which we fixed.
- Security: hashed passwords, token-based sign-in, parameterised SQL queries, and rate limiting. Login never reveals whether a username exists, and a friend request to an unknown username gets a neutral reply.
- Teamwork through Git, with a dev branch and reviewed feature branches, and JSDoc on key classes and methods.

## Traceability

For the traceability part of the demo, we follow two use cases from the requirement to the tests, showing the class diagram, the code in the IDE, and the tests running:

| Step | ViewAlternatives (#3-2) | SearchVenue (#5-1) |
|---|---|---|
| Requirement | Functional requirement #3 (view venue detail), which includes less crowded venues of the same type within 3 km | Functional requirement #5 (search): find venues by name, address or postal code |
| Use case | ViewAlternatives (#3-2), extending ViewVenueDetail (#3-1) | SearchVenue (#5-1) |
| Design | VenueController and AlternativesFinder in the Lab 3 design class diagram | SearchUI and SearchController in the Lab 3 design class diagram |
| Code | server/src/services/alternatives.ts, called by server/src/routes/venues.ts and shown by DetailScreen in the app | app/src/services/venueSearch.ts, used by the Search tab of HomeScreen |
| Tests | Black-box cases BB-01 to BB-13, including the 3 km boundary defect they found | Basis path cases WB2-P1 to P6 for tokenHits, plus the search ranking tests |